

System-level I/O

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

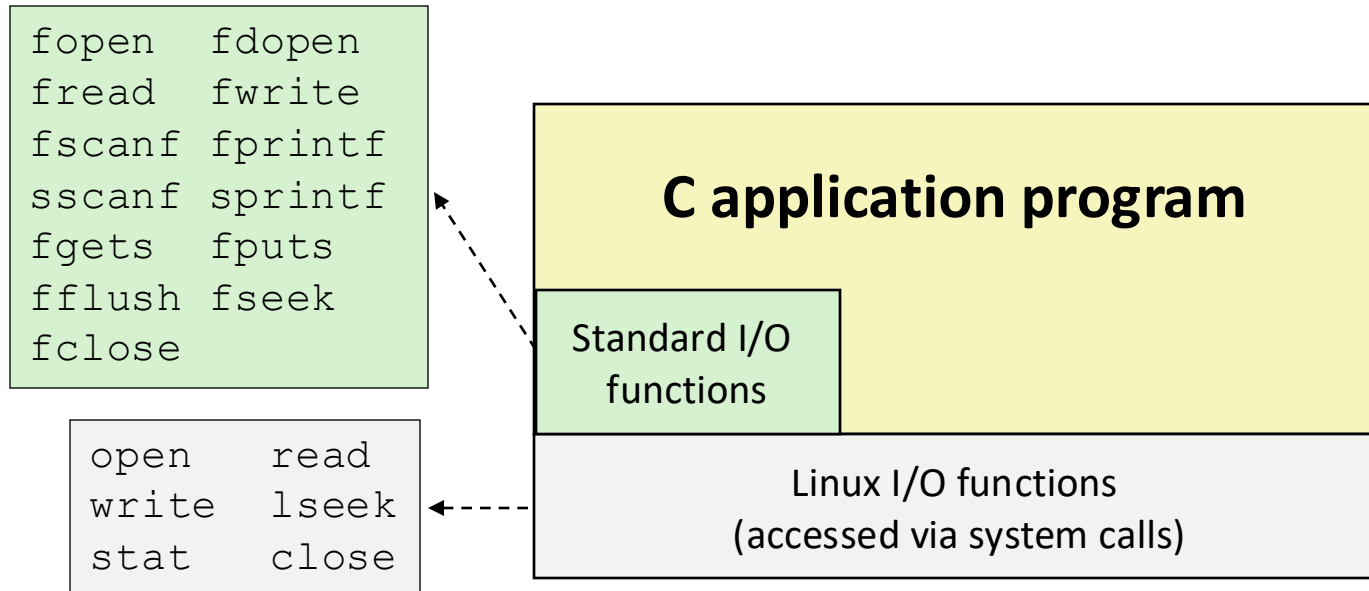
Overview

- Linux I/O
- C Standard I/O

Linux I/O and C Standard I/O

■ Two sets

- System-level and C-level

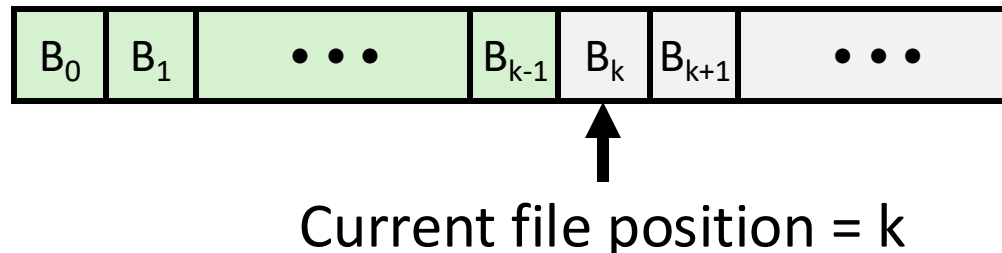


Linux I/O overview

- **A Linux file is a sequence of m bytes**
 - $B_0, B_1, \dots, B_k, \dots, B_{m-1}$
- **All I/O devices are represented as files**
 - `/dev/sda2` (`/usr` disk partition)
 - `/dev/tty2` (terminal)
- **Even the kernel is represented as a file**
 - `/boot/vmlinuz-5.10.0-23-amd64` (kernel image)
 - `/proc` (kernel data structures)

Linux I/O overview

- **Elegant mapping of files to devices allows kernel to export a simple interface:**
 - Opening and closing files
 - `open()` and `close()`
 - Reading and writing a file
 - `read()` and `write()`
 - Changing the current file position (`seek`)
 - Indicates next offset into file to read or write
 - `lseek()`



File types

- **Each file has a type indicating its role in the system**
 - Regular file: Contains arbitrary data
 - Directory: Index for a related group of files

- **Other file types beyond our scope in this document**
 - Socket
 - Named pipes (FIFOs)
 - Symbolic links
 - Character and block devices

Regular files

- A regular file contains arbitrary data
- Applications often distinguish between text files and binary files
 - Text files are regular files with only ASCII or Unicode characters
 - Binary files are everything else
 - e.g., object files, JPEG images
 - Kernel doesn't know the difference!

Text files

■ A text file is sequence of text lines

- A text line is sequence of chars terminated by *newline char* ('`\n`')
 - Newline is `0xa`, same as ASCII line feed character (LF)

■ End of line (EOL) indicators in other systems

- Mac OS: '`\n`' (`0xa`)
 - Line feed (LF)
- Windows and Internet protocols: '`\r\n`' (`0xd 0xa`)
 - Carriage return (CR) followed by line feed (LF)



Directories

■ Directory consists of an array of links

- Each link maps a filename to a file

■ Each directory contains at least two entries

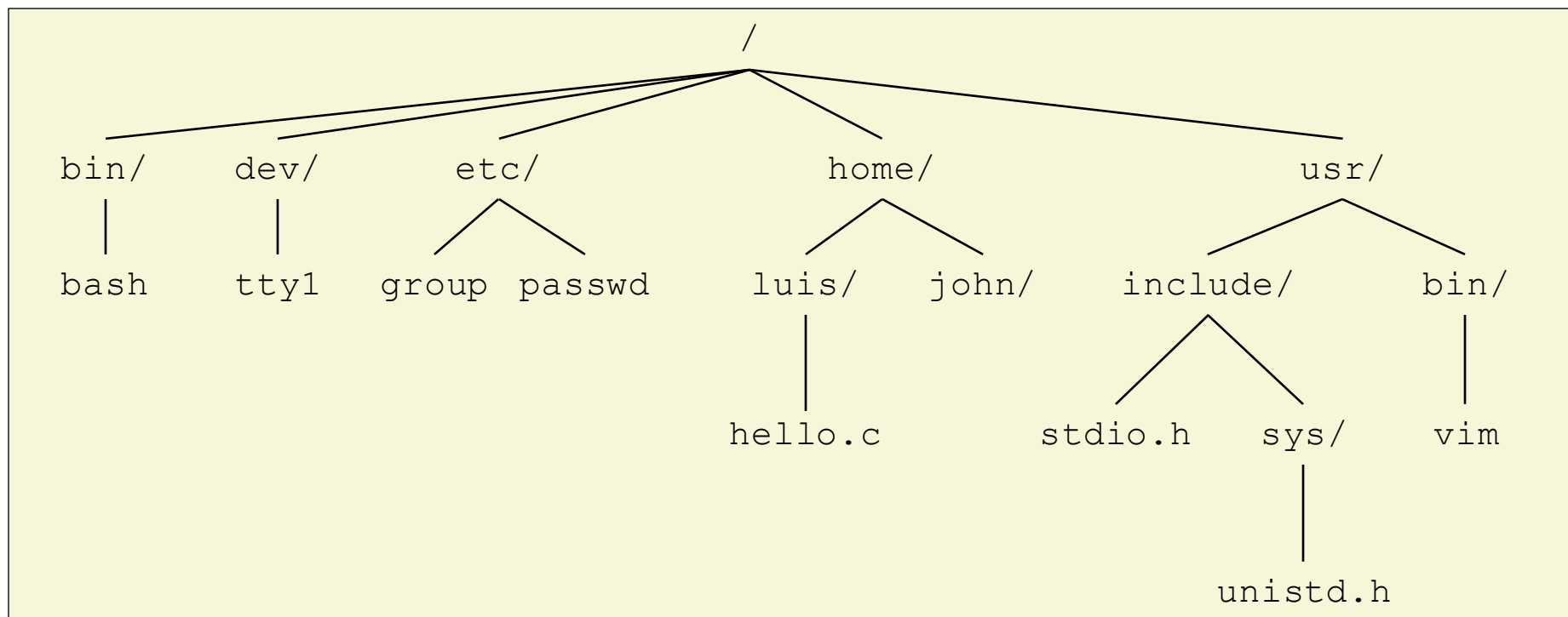
- `.` (dot) is a link to itself
- `..` (dot dot) is a link to the parent directory in the directory hierarchy

■ Commands for manipulating directories

- `mkdir`: create empty directory
- `ls`: view directory contents
- `rmdir`: delete empty directory

Directory hierarchy

- All files are organized as a hierarchy anchored by root directory named `/` (slash)

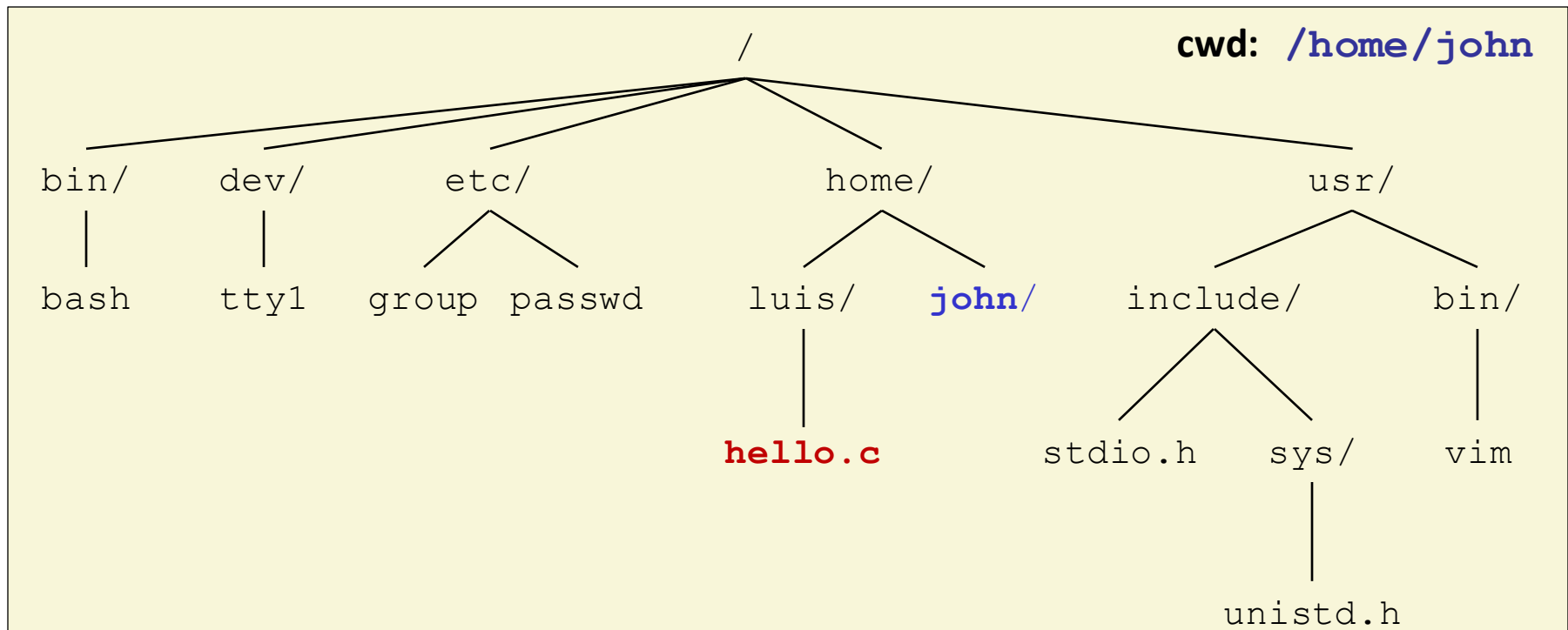


- Kernel maintains current working directory (`cwd`) for each process, modified using the `cd` command

Pathnames

■ Locations of files in the hierarchy denoted by pathnames

- Absolute pathname starts with '/' and denotes path from root
 - `/home/luis/hello.c`
- Relative pathname denotes path from current working directory
 - `../luis/hello.c`



Accessing directories

- **Only recommended operation on a directory: read its entries**
 - `dirent` structure contains information about a directory entry
 - `DIR` structure contains information about directory while stepping through its entries

```
#include <sys/types.h>
#include <dirent.h>
...
DIR *directory;
struct dirent *de;
...
if (!(directory = opendir(dir_name)))
    error("Failed to open directory");
...
while (0 != (de = readdir(directory))) {
    printf("Found file: %s\n", de->d_name);
}
...
closedir(directory);
```

Opening files

- Opening a file informs the kernel that you are getting ready to access that file

```
int fd;    /* file descriptor */

if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {
    perror("open");
    exit(1);
}
```

- Returns a small identifying integer, a **file descriptor**
 - `fd == -1` indicates that an error occurred
- Each process created by a Linux shell begins life with 3 open files associated with a terminal:
 - 0: standard input (`stdin`)
 - 1: standard output (`stdout`)
 - 2: standard error (`stderr`)

Closing files

- Closing a file informs the kernel that you are finished accessing that file

```
int fd;      /* file descriptor */
int retval;  /* return value */

if((retval = close(fd)) < 0) {
    perror("close");
    exit(1);
}
```

- Closing an already closed file is a recipe for disaster in threaded programs (more on this later)
 - Always check return codes, even for seemingly benign functions such as `close()`

Reading files

- Reading a file copies bytes from the current file position to memory, and then updates file position

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open file fd ... */
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
    perror("read");
    exit(1);
}
```

- Returns number of bytes read from file `fd` into `buf`
 - Return type `ssize_t` is signed integer
 - `nbytes < 0` indicates that an error occurred
 - **Short counts** (`nbytes < sizeof(buf)`) are possible and are not errors!

Writing files

- **Writing a file copies bytes from memory to the current file position, and then updates current file position**

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open the file fd ... */
/* Then write up to 512 bytes from buf to file fd */
if ((nbytes = write(fd, buf, sizeof(buf))) < 0) {
    perror("write");
    exit(1);
}
```

- **Returns number of bytes written from `buf` to file `fd`**
 - `nbytes < 0` indicates that an error occurred
 - As with reads, short counts are possible and are not errors!

Simple Linux I/O example

■ Copying file to stdout, one byte at a time

```
int main(int argc, char *argv[]){
    char c;
    int infd = STDIN_FILENO;

    if (argc == 2)
        infd = open(argv[1], O_RDONLY, 0);

    while(read(infd, &c, 1) != 0)
        write(STDOUT_FILENO, &c, 1);
    exit(0);
}
```

showfile1_nobuf.c

■ Demo:

```
linux> strace ./showfile1_nobuf names.txt
```

On short counts

■ Short counts can occur in these situations:

- Encountering (end-of-file) EOF on reads
- Reading text lines from a terminal
- Reading and writing network sockets

■ Short counts never occur in these situations:

- Reading from disk files (except for EOF)
- Writing to disk files

■ Best practice is to always allow for short counts

Buffered I/O example

■ Copying file to stdout, BUFSIZE bytes at a time

```
#define BUFSIZE 64

int main(int argc, char *argv[]){
    char buf[BUFSIZE];
    int infd = STDIN_FILENO;

    if(argc == 2)
        infd = open(argv[1], O_RDONLY, 0);

    while((nread = read(infd, buf, BUFSIZE)) != 0)
        write(STDOUT_FILENO, buf, nread);
    exit(0);
}
```

showfile2_buf.c

■ Demo:

```
linux> strace ./showfile2_buf names.txt
```

File metadata

■ Per-file metadata maintained by kernel

- Accessed by users with the `stat` and `fstat` functions

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t      st_dev;      /* Device */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* Protection and file type */
    nlink_t    st_nlink;    /* Number of hard links */
    uid_t      st_uid;      /* User ID of owner */
    gid_t      st_gid;      /* Group ID of owner */
    dev_t      st_rdev;     /* Device type (if inode device) */
    off_t      st_size;     /* Total size, in bytes */
    unsigned long st_blksize; /* Blocksize for filesystem I/O */
    unsigned long st_blocks; /* Number of blocks allocated */
    time_t     st_atime;     /* Time of last access */
    time_t     st_mtime;     /* Time of last modification */
    time_t     st_ctime;     /* Time of last change */
};
```

Example of accessing file metadata

```
int main(int argc, char **argv){  
    struct stat stat;  
    char *type, *readok;
```

```
    stat(argv[1], &stat);  
    if (S_ISREG(stat.st_mode))          /* Determine file type */  
        type = "regular";  
    else if (S_ISDIR(stat.st_mode))  
        type = "directory";  
    else  
        type = "other";  
    if((stat.st_mode & S_IRUSR)) /* Check read access */  
        readok = "yes";  
    else  
        readok = "no";
```

```
    printf("type: %s, read: %s\n", type, readok);  
    exit(0);
```

```
}
```

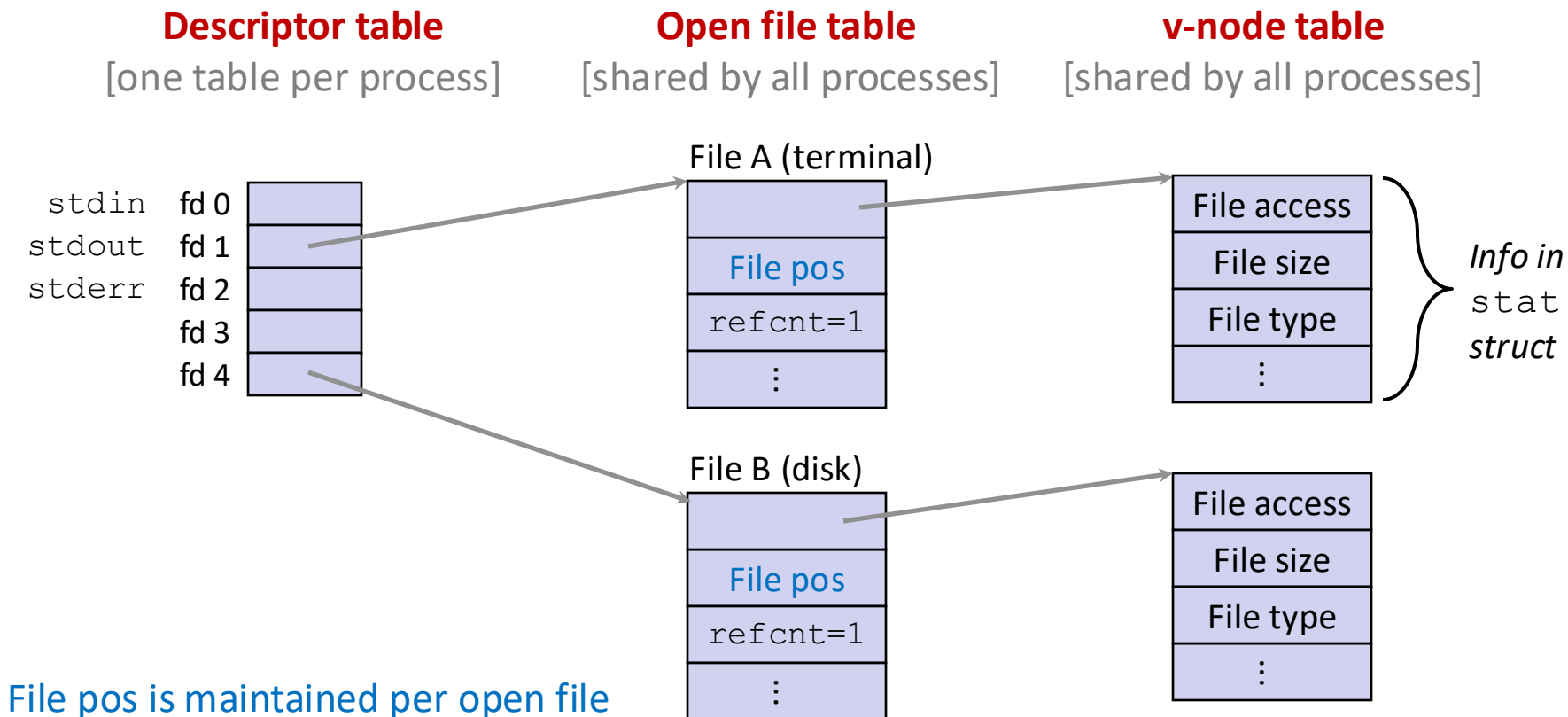
statcheck.c

```
linux> ./statcheck statcheck.c  
type: regular, read: yes  
linux> chmod 000 statcheck.c  
linux> ./statcheck statcheck.c  
type: regular, read: no  
linux> ./statcheck ..  
type: directory, read: yes
```

How the Linux kernel represents open files

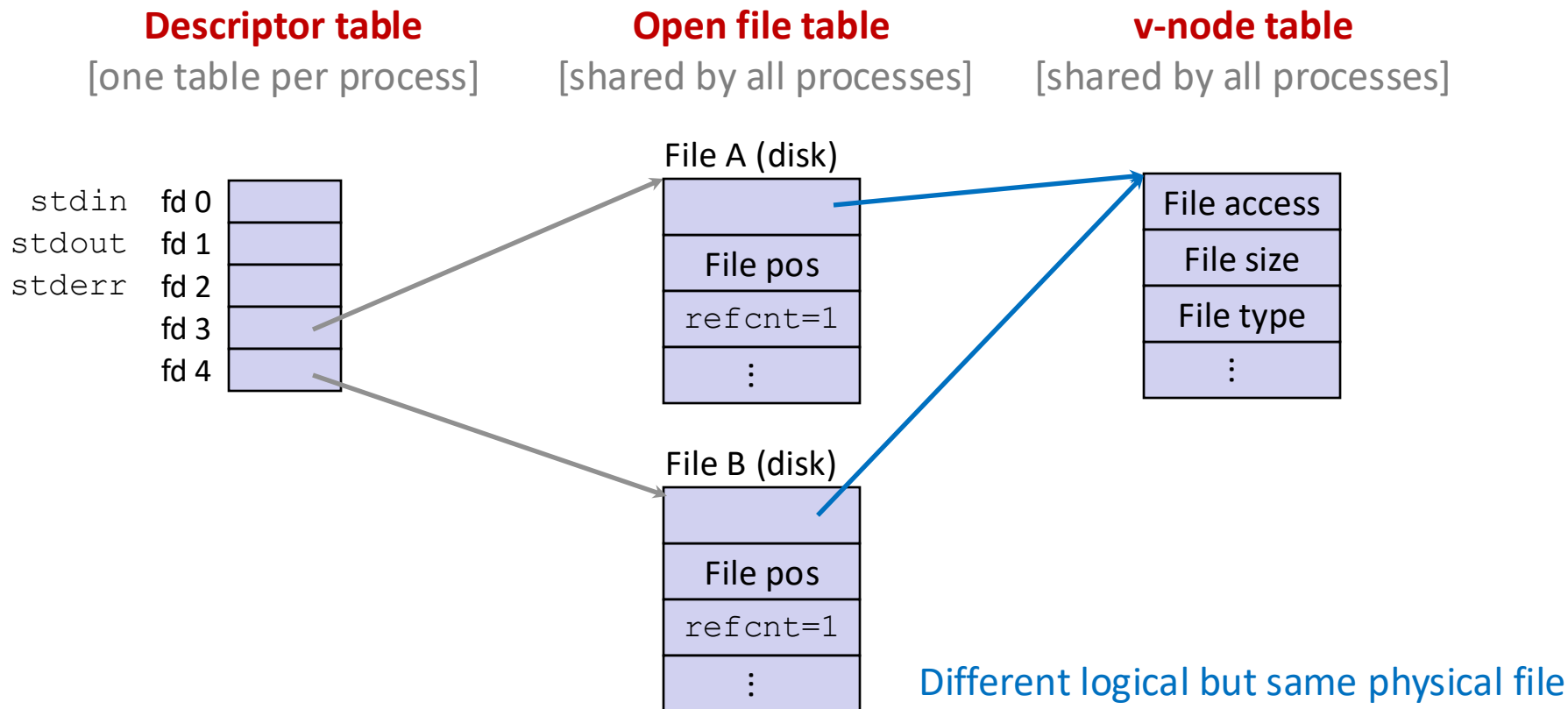
■ E.g., two descriptors referencing two distinct open files

- Descriptor 1 (`stdout`) points to terminal, and descriptor 4 points to open disk file



File sharing

- E.g., two distinct descriptors sharing the same disk file through two distinct open file table entries
 - e.g., calling `open` twice with the same `filename` argument

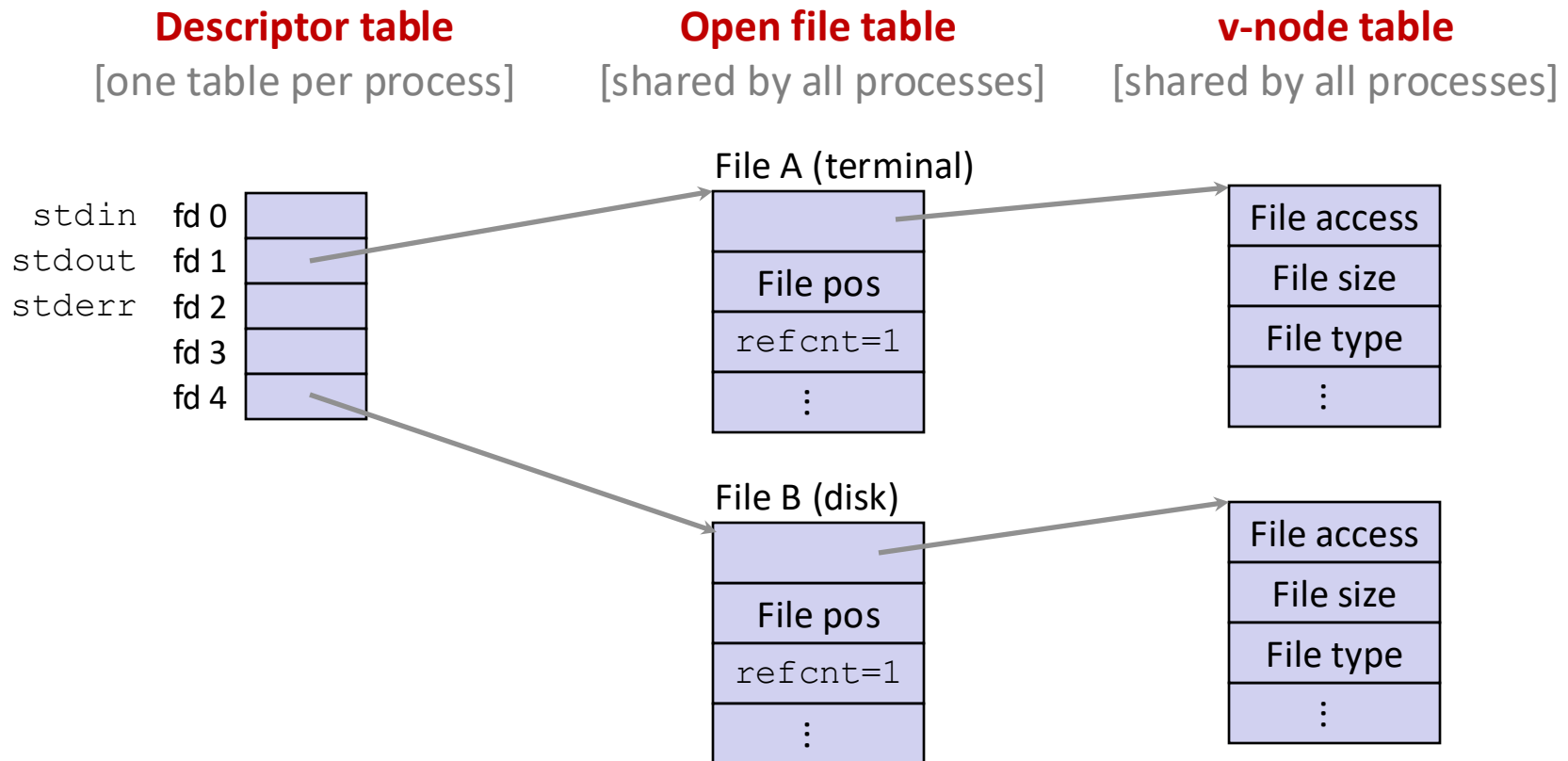


How processes share files: `fork`

■ A child process inherits its parent's open files

- Note: situation unchanged by `exec` functions (use `fcntl` to change)

■ Before `fork()` call:



How processes share files: `fork`

- A child process inherits its parent's open files
- **After** `fork()` :
 - Child's table same as parent's, and +1 to each `refcnt`

Descriptor table

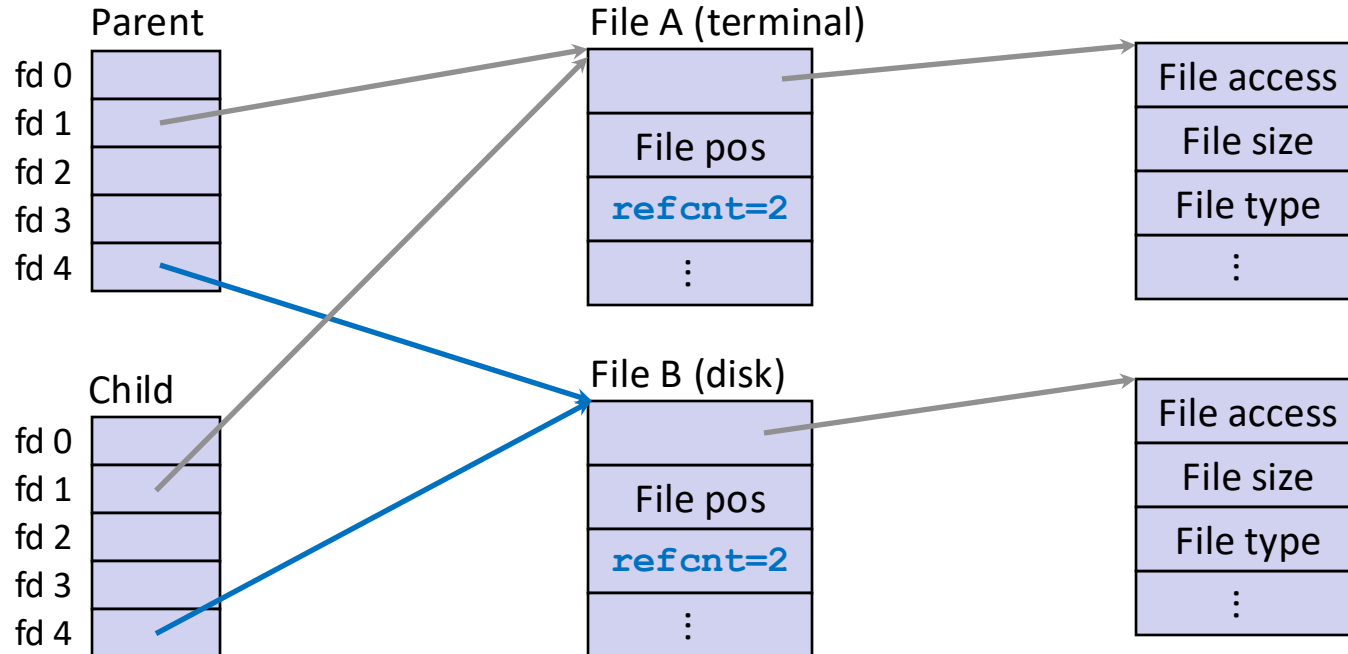
[one table per process]

Open file table

[shared by all processes]

v-node table

[shared by all processes]



File is shared between processes

I/O redirection

■ Question: How does a shell implement I/O redirection?

```
linux> ls > foo.txt
```

■ Answer: By calling the `dup2 (oldfd, newfd)` function

- Copies (per-process) descriptor table entry `oldfd` to entry `newfd`

Descriptor table
before `dup2 (4, 1)`

fd 0	
fd 1	a
fd 2	
fd 3	
fd 4	b



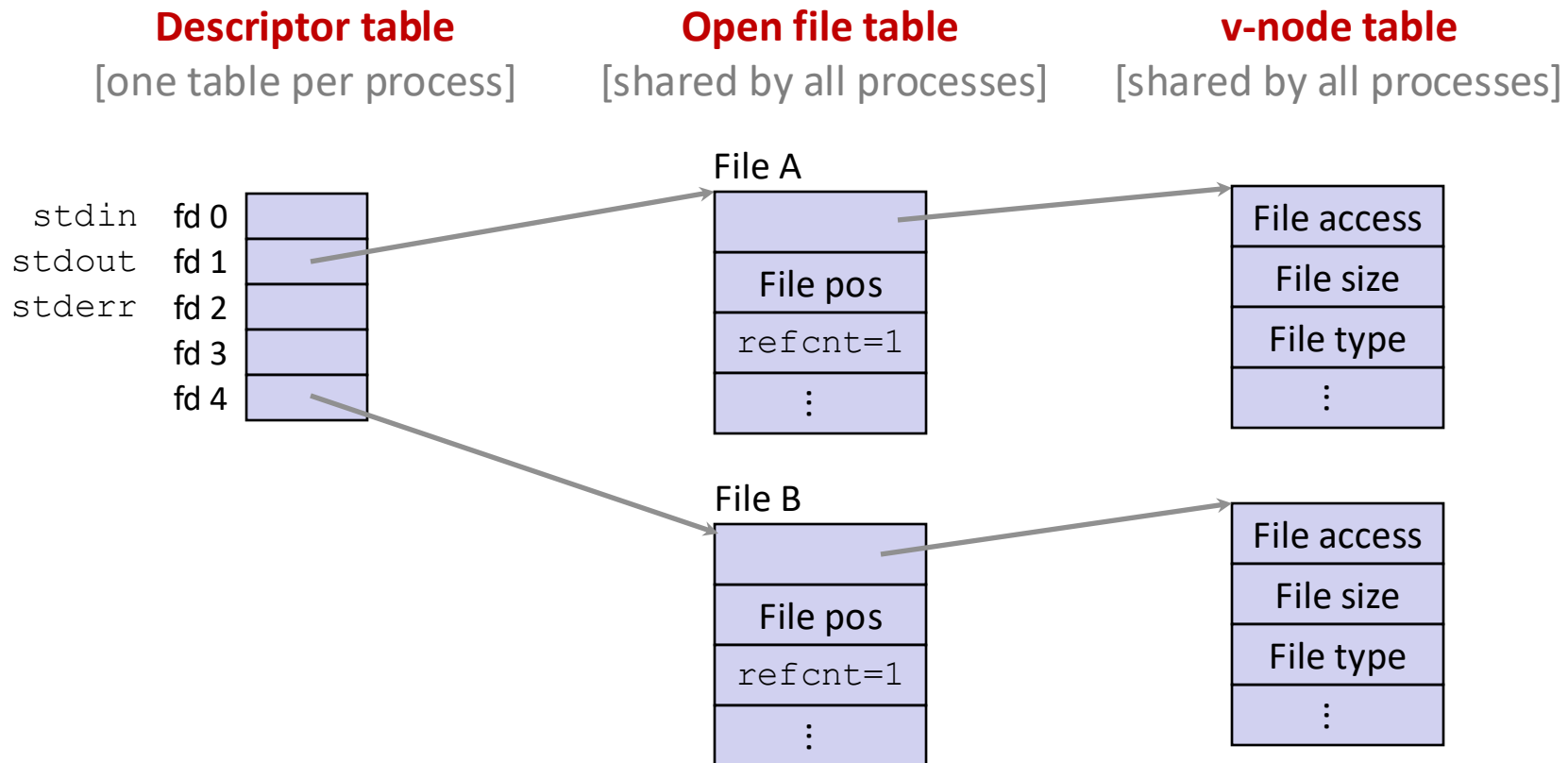
Descriptor table
after `dup2 (4, 1)`

fd 0	
fd 1	b
fd 2	
fd 3	
fd 4	b

I/O redirection example

■ Step #1: open file to which `stdout` should be redirected

- Happens in child executing shell code, before `exec`



I/O redirection example (cont.)

■ Step #2: call `dup2 (4 , 1)`

- Cause `fd=1` (`stdout`) to refer to disk file pointed at by `fd=4`

Descriptor table

[one table per process]

stdin	fd 0	
stdout	fd 1	
stderr	fd 2	
	fd 3	
	fd 4	

Open file table

[shared by all processes]

File A

File pos
refcnt=0
⋮

File B

File pos
refcnt=2
⋮

v-node table

[shared by all processes]

File access
File size
File type
⋮

File access
File size
File type
⋮

Two descriptors point to the same file

Exercise 1: I/O and redirection

- What would this program print for a file containing “abcde”?

```
int main(int argc, char *argv[]){
    int fd1, fd2, fd3;
    char c1, c2, c3;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    fd2 = open(fname, O_RDONLY, 0);
    fd3 = open(fname, O_RDONLY, 0);
    dup2(fd2, fd3);
    read(fd1, &c1, 1);
    read(fd2, &c2, 1);
    read(fd3, &c3, 1);
    printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
    return 0;
}
```

Exercise 1: I/O and redirection

- What would this program print for a file containing “abcde”?

```
int main(int argc, char *argv[]){
    int fd1, fd2, fd3;
    char c1, c2, c3;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    fd2 = open(fname, O_RDONLY, 0);
    fd3 = open(fname, O_RDONLY, 0);
    dup2(fd2, fd3);
    read(fd1, &c1, 1);
    read(fd2, &c2, 1);
    read(fd3, &c3, 1);
    printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
    return 0;
}
```

← dup2(oldfd, newfd)

c1 = a, c2 = a, c3 = b

Exercise 2: Process control and I/O

- What would this program print for a file containing “abcde”?

```
int main(int argc, char *argv[]){
    int fd1;
    int s = getpid() & 0x1;
    char c1, c2;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    read(fd1, &c1, 1);
    if(fork()){    /* Parent */
        sleep(s);
        read(fd1, &c2, 1);
        printf("Parent: c1 = %c, c2 = %c\n", c1, c2);
    }else{        /* Child */
        sleep(1-s);
        read(fd1, &c2, 1);
        printf("Child: c1 = %c, c2 = %c\n", c1, c2);
    }
    return 0;
}
```

Exercise 2: Process control and I/O

- What would this program print for a file containing “abcde”?

```
int main(int argc, char *argv[]){
    int fd1;
    int s = getpid() & 0x1;
    char c1, c2;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    read(fd1, &c1, 1);
    if(fork()){ /* Parent */
        sleep(s);
        read(fd1, &c2, 1);
        printf("Parent: c1 = %c, c2 = %c\n", c1, c2);
    }else{ /* Child */
        sleep(1-s);
        read(fd1, &c2, 1);
        printf("Child: c1 = %c, c2 = %c\n", c1, c2);
    }
    return 0;
}
```

Child : c1 = a, c2 = b
Parent: c1 = a, c2 = c

Parent: c1 = a, c2 = b
Child : c1 = a, c2 = c

Standard I/O functions

- The C standard library (`libc.so`) contains a collection of higher-level **standard I/O** functions
 - Documented in Appendix B of Brian Kernighan and Dennis Ritchie' book
- Examples of standard I/O functions:
 - Opening and closing files (`fopen` and `fclose`)
 - Reading and writing bytes (`fread` and `fwrite`)
 - Reading and writing text lines (`fgets` and `fputs`)
 - Formatted reading and writing (`fscanf` and `fprintf`)

Standard I/O streams

■ Standard I/O models open files as **streams**

- Abstraction for a file descriptor and a buffer in memory

■ C programs begin life with three open streams defined in **stdio.h**

- `stdin` (standard input)
- `stdout` (standard output)
- `stderr` (standard error)

```
#include <stdio.h>
extern FILE *stdin; /* standard input (descriptor 0) */
extern FILE *stdout; /* standard output (descriptor 1) */
extern FILE *stderr; /* standard error (descriptor 2) */

int main(){
    fprintf(stdout, "Hello, world\n");
}
```

Buffered I/O: motivation

- **Applications often read/write one character at a time**
 - `getc`, `putc`, `ungetc`
 - `gets`, `fgets`
 - Read line of text one character at a time, stopping at newline
- **Implementing as Linux I/O calls expensive**
 - `read` and `write` require Linux kernel calls
 - > 10,000 clock cycles

Buffered I/O: motivation

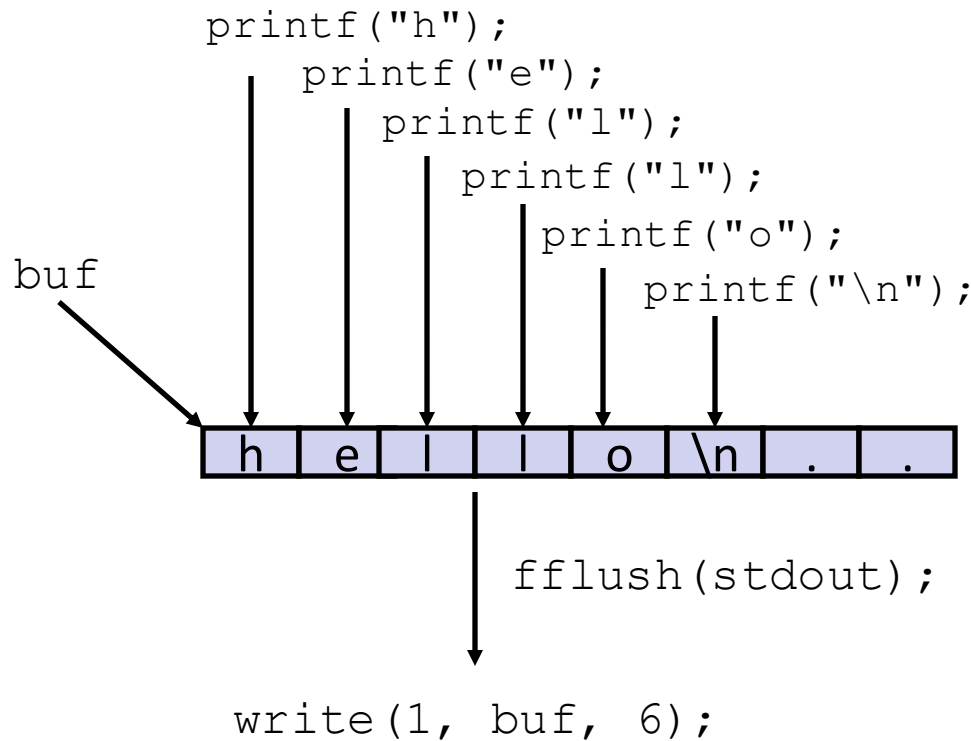
■ Solution: Buffered read

- Use Unix `read` to grab block of bytes
- User input functions take one byte at a time from buffer
 - Refill buffer when empty



Buffering in standard I/O

- Standard I/O functions use buffered I/O



- Buffer flushed to output fd on “\n”, call to `fflush` or `exit`, or return from `main`

Standard I/O Buffering in Action

- You can see this buffering in action for yourself, using the always fascinating Linux `strace` program

```
#include <stdio.h>

int main(){
    printf("h");
    printf("e");
    printf("l");
    printf("l");
    printf("o");
    printf("\n");
    fflush(stdout);
    exit(0);
}
```

```
linux> strace ./hello
execve("./hello", ["hello"], [/* ... */]).
...
write(1, "hello\n", 6)                = 6
...
exit_group(0)                         = ?
```

Standard I/O example

■ Copying file to stdout, line-by-line with stdio

```
#define MLINE 1024

int main(int argc, char *argv[]){
    char buf[MLINE];
    FILE *infile = stdin;
    if(argc == 2){
        infile = fopen(argv[1], "r");
        if (!infile)
            exit(1);
    }
    while(fgets(buf, MLINE, infile) != NULL)
        fputs(buf, stdout);
    exit(0);
}
```

showfile3_stdio.c

■ Demo:

```
linux> strace ./showfile3_stdio names.txt
```

Another standard I/O example

■ Copying file to stdout, loading entire file with mmap

```
int main(int argc, char **argv){
    struct stat stat;
    if (argc != 2)
        exit(1);
    int infd = open(argv[1], O_RDONLY, 0);
    fstat(infd, &stat);
    size_t size = stat.st_size;
    char *bufp = mmap(NULL, size, PROT_READ, MAP_PRIVATE,
                      infd, 0);
    write(1, bufp, size);
    exit(0);
}
```

showfile4_mmap.c

■ Demo:

```
linux> strace ./showfile4_mmap names.txt
```


Pros and Cons of Linux I/O

■ Pros

- Linux I/O is the most general and lowest overhead form of I/O
 - All other I/O packages are implemented using Linux I/O functions
- Linux I/O provides functions for accessing file metadata
- Linux I/O functions are async-signal-safe and can be used safely in signal handlers

■ Cons

- Dealing with short counts is tricky and error prone
- Efficient reading of text lines requires some form of buffering, also tricky and error prone
- Both issues are addressed by the standard I/O package

Pros and Cons of Standard I/O

■ Pros

- Buffering increases efficiency by decreasing the number of `read` and `write` system calls
- Short counts are handled automatically

■ Cons

- Provides no function for accessing file metadata
- Standard I/O functions are not async-signal-safe, and not appropriate for signal handlers

For further information

- **W. Richard Stevens and Stephen A. Rago, “Advanced Programming in the Unix Environment”, 3rd Edition, Addison Wesley, 2013**
 - Updated from Stevens’s 1993 classic text

- **Michael Kerrisk, “The Linux Programming Interface”, No Starch Press, 2010**
 - Encyclopedic and authoritative